EV POWERTRAIN SIMULATOR

Part 3: Battery Management System

Revision Notes

Thevenin battery model | EKF state-of-charge estimation | Passive cell balancing | Protection logic

Contents

Glossary
1. Thevenin Battery Model
2. SOC Estimation with an Extended Kalman Filter (EKF)
3. Passive Cell Balancing
4. Protection Logic
5. How the Four Parts Fit Together
6. Quick-Revision Formula Sheet
7. Self-Test Questions

These notes explain every concept used in Part 3 of the EV Powertrain Simulator, starting from zero. They include the doubts that came up while learning (shown as **Q.** headings) with worked numerical answers, so a reader can follow all four sub-topics without any other source.

The four sub-topics

#	Sub-topic	Script	One-line purpose
1	Thevenin battery model	`thevenin_model.m`	Predict cell voltage from current and SOC
2	EKF for SOC estimation	`ekf_soc.m`	Estimate the hidden SOC from noisy sensors
3	Passive cell balancing	`cell_balancing.m`	Keep the cells in a pack equally charged
4	Protection logic	`bms_protection.m`	Cut or reduce current when limits are crossed

Important: all cell parameters (R0, R1, C1, Cth, Rth) and the OCV table are **placeholders** shaped like an LFP cell, used to learn the methods. For a real pack, replace them with datasheet or measured values.

Glossary

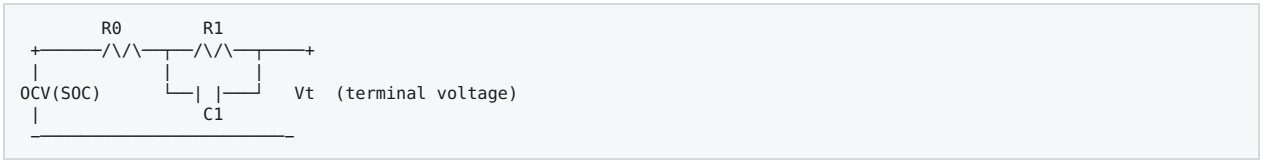
Term	Meaning
Cell / pack	A pack is many cells connected in series. The same current flows through all of them.
SOC (State of Charge)	How full the cell is. 1.0 = 100% full, 0 = empty.
OCV (Open-Circuit Voltage)	The cell's voltage when no current flows. It depends on SOC.
Terminal voltage, Vt	The voltage you actually measure at the cell's terminals while current flows.
R0	Internal resistance. Causes an instant voltage change when current changes.
R1, C1, τ	A resistor-capacitor pair that models the slow part of the voltage response. `τ = R1 × C1` is its time constant.
U1	The voltage across the RC pair.
Coulomb counting	Estimating SOC by adding up the current over time.
EKF	Extended Kalman Filter: an algorithm that combines a model prediction with a sensor measurement to estimate hidden quantities.
P	The EKF's uncertainty about its own estimate.
Qn	How much the EKF distrusts the model (process noise).
Rn	How much the EKF distrusts the voltage sensor (measurement noise).
K (Kalman gain)	How strongly the EKF corrects itself using the sensor.
H	How sensitive voltage is to each state. For SOC, it is the slope of the OCV curve.
Innovation	The "surprise": measured voltage minus predicted voltage.
Spread	Highest SOC minus lowest SOC among the cells of a pack.
Bleeding	Slowly draining a cell through a resistor to balance it.
Dead band	A small tolerance zone in which the controller does nothing, to avoid constant switching.
Derating	Gradually reducing the allowed current as a limit is approached.
Latched fault	A fault that stays active until someone resets it, even if the cause disappears.
Contactor	The main switch that connects the battery to the load or charger.
Cth	Thermal capacitance (J/K): how much heat the cell absorbs per degree of temperature rise.
Rth	Thermal resistance (K/W): how hard it is for heat to escape to the surroundings.

Sign convention used everywhere: current is **positive when discharging** (flowing out of the battery, like driving) and **negative when charging** (flowing in).

1. Thevenin Battery Model

1.1 What it is

A battery is not a simple voltage source. When current flows, its voltage changes in two ways: instantly, and slowly. The Thevenin equivalent circuit captures both with three elements.



Element	What it models
OCV(SOC)	The ideal voltage at the current charge level
R0	**Instant** voltage drop when current starts or stops
R1 + C1	**Slow** sag and recovery of voltage after a change in current

1.2 The three equations

Time step `dt`, discharge current `I > 0`.

```
1. SOC(k+1) = SOC(k) - (η * dt / Q) * I(k)
2. U1(k+1)  = exp(-dt/τ) * U1(k) + R1 * (1 - exp(-dt/τ)) * I(k)      τ = R1*C1
3. Vt(k)    = OCV(SOC(k)) - I(k)*R0 - U1(k)
```

- Equations 1 and 2 are the **state update** (prediction): how SOC and U1 evolve.
- Equation 3 is the **output**: the voltage a sensor would read.
- `Q` is capacity in ampere-seconds (60 Ah × 3600 = 216,000 A·s). `η` is coulombic efficiency (1 here).

1.3 Parameters used (placeholders)

Parameter	Value
Capacity Q	60 Ah = 216,000 A·s
R0	0.002 Ω
R1	0.001 Ω
C1	20,000 F
τ = R1·C1	20 s
dt	1 s

OCV table (LFP-like shape):

SOC	0	0.1	0.2	0.5	0.8	0.9	1.0
OCV (V)	2.80	3.20	3.28	3.30	3.33	3.38	3.45

1.4 Core code

```
OCV = @(s) interp1(soc_tbl, ocv_tbl, s, 'linear', 'extrap');

for k = 1:N-1
    Vt(k)    = OCV(soc(k)) - I(k)*R0 - U1(k);
    soc(k+1) = soc(k) - eta*dt/Q * I(k);
    U1(k+1)  = exp(-dt/tau)*U1(k) + R1*(1-exp(-dt/tau))*I(k);
end
Vt(N) = OCV(soc(N)) - I(N)*R0 - U1(N);
```

1.5 Test profile and how to read the graph

Current: 60 A base load, 150 A burst from 300 to 600 s, 0 A rest from 900 to 1000 s.

Time	Current	What Vt does	Why
0 s	60 A starts	Quick drop, then slow sag	R0 (instant), RC pair (slow)
300 s	up to 150 A	Big instant drop, then sags to about 2.87 V by 600 s	More current means more voltage lost across R0 and R1
600 s	back to 60 A	Jumps up, then settles	R0 loss shrinks instantly, RC recovers slowly
900 s	0 A (rest)	Jumps up, then creeps toward OCV	No current, no losses
1000 s	60 A returns	Drops again	Load is back

SOC falls smoothly from 1.0 to about 0.4 because the profile uses 129,000 A·s out of 216,000 A·s (60%):

```
0-300 s:     60 A × 300 s  = 18,000 A·s
300-600 s:  150 A × 300 s  = 45,000 A·s
600-900 s:   60 A × 300 s  = 18,000 A·s
900-1000 s:   0 A          =      0
1000-1800 s: 60 A × 800 s  = 48,000 A·s
Total = 129,000 A·s  →  129,000 / 216,000 = 0.6  →  SOC 1.0 → 0.4
```

Q. What do "sag" and "slow" mean?

After the current switches on, Vt does **two** things:

```
Vt
 |________\
 |          |   ← 1. instant drop (vertical line)  = R0
 |           \
 |            \_____________   ← 2. slow sag (curve) = R1 and C1
 +---------------------------- time
        300 s
```

- **Sag** = the voltage keeps falling gradually after the first drop.
- **Slow** = it takes tens to hundreds of seconds, not an instant.
- Analogy: pushing a spring-loaded door. You feel resistance immediately (R0), and it gets harder the longer you hold it open (the RC part).
- The same happens in reverse when the current drops: an instant jump up, then a slow recovery.

Q. What is 0.002?

`R0 = 0.002` is the internal resistance in ohms (2 milliohms). Voltage lost across it is `current × R0`:

Current	Voltage drop across R0
60 A	0.12 V
80 A	0.16 V
120 A	0.24 V

It also makes heat: `heat = I² x R0` (80 A gives 12.8 W). While discharging, Vt = OCV minus the drop (voltage falls). While charging, Vt = OCV plus the drop (voltage rises).

Q. How do I get the voltage from the SOC?

Use the OCV table and **straight-line interpolation** between the two nearest points:

```
V = V_low + (SOC - SOC_low) * (V_high - V_low) / (SOC_high - SOC_low)
```

SOC	Nearest points	Slope	Voltage
0.98	0.90 → 3.38, 1.00 → 3.45	0.7 V per unit SOC	3.38 + 0.08 × 0.7 = **3.436 V**
0.95	same	0.7	3.38 + 0.05 × 0.7 = **3.415 V**
0.92	same	0.7	3.38 + 0.02 × 0.7 = **3.394 V**
0.60	0.50 → 3.30, 0.80 → 3.33	0.1	3.30 + 0.10 × 0.1 = **3.31 V**

In MATLAB, `interp1` does this. In real life, the table comes from the cell datasheet or from a slow lab test (for example C/20 with rest periods). Going the other way (voltage to SOC) uses the same table backwards.

Q. Where does the current profile come from in my Simulink model?

Take the battery current signal from the Part 1 model (the current leaving the battery or the DC bus current) into a **To Workspace** block with variable name `I_batt` and format **Timeseries**. Then in the script: `I = I_batt.Data(:); t = I_batt.Time(:); dt = t(2) - t(1);`. Make sure discharge is positive.

Key takeaways (Section 1)

- R0 gives the **instant** voltage change, the RC pair gives the **slow** one.
- Equations 1 and 2 predict the state. Equation 3 predicts the voltage.
- The LFP OCV curve is very flat in the middle, which matters a lot in Section 2.

2. SOC Estimation with an Extended Kalman Filter (EKF)

2.1 Why SOC must be estimated

No sensor measures SOC. A BMS can only measure voltage, current and temperature, and must **estimate** SOC from them. The dashboard's "62% battery, 210 km range" is such an estimate.

Without a good SOC estimate	What goes wrong
Wrong range	The car says 80 km left but stops at 50 km
Battery damage	Over-discharge shortens cell life
Unused capacity	A too-cautious BMS hides 10 to 15% of the battery
Bad charging	The charger can't tell when to taper
Bad balancing	Decisions depend on each cell's SOC
Safety	Over-charge and over-discharge are fire risks

2.2 Coulomb counting vs. EKF

Coulomb counting adds up the current: `SOC = SOC_start - (current × time) / capacity`.

	Coulomb counting	EKF
Needs a battery model	No	Yes
Needs a voltage sensor	No	Yes
Recovers from a wrong start	**No**	**Yes**
Drifts over time	**Yes** (sensor errors pile up)	No (voltage anchors it)
Sensitive to current-sensor bias	High	Low
Computation	Tiny	Moderate
Tuning needed	None	Qn, Rn, P
Sensitive to model errors	No	**Yes**

Analogy: walking in a dark room. Counting steps (Coulomb counting) drifts, because each step is slightly off. Touching a wall (the voltage measurement) is noisy but never drifts. The EKF blends both and trusts each in proportion to its reliability.

The EKF **contains** Coulomb counting: its predict step is Coulomb counting, and its correct step adds feedback from the voltage.

2.3 What "Extended" means

The basic Kalman filter works only for straight-line (linear) systems. The OCV-SOC curve is curved, so the EKF replaces the curve with its **tangent line at the current estimate** at every step. That is all "extended" means.

2.4 The EKF loop (every time step)

Step	What happens	In the battery
1. Predict	Use the model to guess the next state	SOC by Coulomb counting, plus the RC voltage
2. Measure	Read the real sensor	Actual terminal voltage
3. Compare	Find the error between predicted and measured voltage	"Model says 3.30 V, sensor says 3.28 V"
4. Correct	Nudge the estimate by (error × gain)	SOC is adjusted slightly

States: `x = [SOC; U1]`

Predict:

```
x_pred = A*x + B*I          A = [1 0; 0 exp(-dt/τ)]
P_pred = A*P*A' + Qn        B = [-η*dt/Q; R1*(1-exp(-dt/τ))]
```

Correct:

```
Vt_pred = OCV(SOC_pred) - I*R0 - U1_pred
innov   = Vt_measured - Vt_pred
H       = [dOCV/dSOC, -1]
K       = P_pred*H' / (H*P_pred*H' + Rn)
x       = x_pred + K*innov
P       = (I - K*H) * P_pred
```

2.5 The symbols in plain words

Symbol	Meaning
P	How unsure we are (big P means "not sure")
Qn	How much we distrust the model
Rn	How much we distrust the voltage sensor
K	How much of the "surprise" to believe (0 = ignore the sensor)
H	How much the voltage tells us about each state. For SOC it is the **slope of the OCV curve**.

2.6 Settings used in the test

Setting	Value
Voltage sensor	true voltage + 5 mV noise
Current sensor	1.02 × true current (**+2% bias**) + 0.5 A noise
Initial guess	SOC deliberately wrong (see tests)
`P0`	`diag([0.05, 1e-4])`
`Qn`	`diag([1e-9, 1e-7])`
`Rn`	`0.005²`

2.7 Core code

```
for k = 2:N
    % --- PREDICT ---
    x = A*x + B*Im(k-1);
    P = A*P*A' + Qn;
    soc_cc(k) = soc_cc(k-1) - eta*dt/Q*Im(k-1);   % plain Coulomb counting

    % --- CORRECT ---
    ds   = 1e-3;
    dOCV = (OCV(x(1)+ds) - OCV(x(1)-ds)) / (2*ds); % slope of OCV curve
    H    = [dOCV, -1];
    Vt_pred = OCV(x(1)) - Im(k)*R0 - x(2);
    innov   = Vm(k) - Vt_pred;

    S = H*P*H' + Rn;
    K = P*H'/S;
    x = x + K*innov;
    x(1) = min(max(x(1), 0), 1);                  % keep SOC in [0, 1]
    P = (eye(2) - K*H)*P;
    soc_ekf(k) = x(1);
end
```

Line-by-line:

Line	Meaning
`x = A*x + B*Im(k-1)`	Run the Thevenin state equations one step forward
`P = A*P*A' + Qn`	Uncertainty grows, because predicting always adds doubt
`dOCV`	Slope of the OCV curve at the current SOC estimate (this is H)
`Vt_pred`	The voltage the model expects right now
`innov`	The "surprise": sensor minus prediction
`S`	Total uncertainty of that surprise
`K = P*H'/S`	Kalman gain
`x = x + K*innov`	The correction
`P = (eye(2)-K*H)*P`	Uncertainty shrinks, because we learned something

2.8 Test 1: start near full charge (steep OCV region)

True SOC starts at 1.0, the EKF and Coulomb counting start at 0.8 (a 0.2 error).

	Result
Coulomb counting	Stays about 0.2 below the truth for the whole run (error about -0.2, slightly worse at the end)
EKF	Jumps to the true value within seconds, error about 0.005 afterwards
RMSE	About 0.2 (Coulomb counting) vs. about 0.01 (EKF)

Why so fast: near 100% SOC the OCV curve is steep, so voltage tells the EKF a lot about SOC.

2.9 Test 2: flat LFP region

True SOC starts at 0.8, the EKF and Coulomb counting start at 0.6. The drive uses 60% of the capacity, so the true SOC goes from 0.8 to about 0.2, entirely in the flat region.

Time	EKF error	What happens
0 s	-0.2	Deliberate wrong start
0 to about 40 s	-0.2 to about 0	Fast correction (P is large, so the EKF trusts the voltage)
after 40 s	slowly rises to about **+0.04** and stays	A small offset the EKF cannot remove

Coulomb counting stays at about -0.2 the whole time. The EKF still removes about 80% of the starting error even in the hardest region.

> **Pitfall:** the drive profile uses 60% of the capacity. If the true SOC starts below about 0.65, the true SOC would go **below zero**, which is impossible. Start the true SOC at 0.8 or higher for this profile.

Q. What does EKF mean?

Extended Kalman Filter. It estimates SOC, which cannot be measured, by combining a model prediction (Coulomb counting) with the voltage sensor. When the predicted and measured voltages disagree, it adjusts SOC to reduce the gap. "Extended" only means it handles curved relationships such as the OCV curve.

Q. Why does the EKF struggle on LFP at about 50% SOC?

The EKF corrects using the **slope of the OCV curve** (H). LFP is very flat in the middle:

SOC range	Voltage change	Slope (V per unit SOC)	Voltage change per 1% SOC
0 to 10%	2.80 to 3.20	4.0	40 mV
10 to 20%	3.20 to 3.28	0.8	8 mV
20 to 50%	3.28 to 3.30	0.07	0.7 mV
50 to 80%	3.30 to 3.33	0.10	1 mV
80 to 90%	3.33 to 3.38	0.5	5 mV
90 to 100%	3.38 to 3.45	0.7	7 mV

The voltage sensor noise is about 5 mV. At 50% SOC a 1% SOC change moves the voltage by only about 0.7 mV, far below the noise, so the EKF cannot tell "SOC changed" from "sensor jitter". At 95% it moves by 7 mV, which is clearly visible.

```
Flat region:  H is tiny  →  K is tiny  →  EKF barely corrects (acts like Coulomb counting)
Steep region: H is big   →  K is big   →  EKF corrects strongly
```

The EKF is not broken in the flat zone. It correctly realises the voltage tells it almost nothing and falls back on the model.

Why LFP is flat: lithium moves between two stable crystal phases, and the voltage stays almost constant while both phases are present (like water boiling at a fixed temperature while heat is added). Real LFP is even flatter than the placeholder table and also shows hysteresis (the voltage at a given SOC differs after charging and discharging).

	NMC/NCA (sloped curve)	LFP (flat curve)
Voltage tells SOC?	Yes, across the range	Only near the ends
EKF correction	Good everywhere	Strong at the ends, weak in the middle
Typical fixes	Standard EKF	Accurate current sensing, hysteresis model, recalibration at the ends

LFP is still widely used because it is safer, lasts longer and costs less.

Q. Where does the leftover offset of +0.04 come from, and why does the EKF stop there?

Short answer: the EKF stops correcting when its predicted voltage **equals** the measured voltage. That happens at a slightly wrong SOC, because the current sensor has a +2% bias and the OCV curve is flat.

Numbers at 60 A, true SOC = 0.50:

```
Real battery (true current 60 A):
  Vt = 3.30 - 60*0.002 - 60*0.001 = 3.120 V

EKF's prediction with the biased current (61.2 A), if it also guesses SOC = 0.50:
  Vt = 3.30 - 61.2*0.002 - 61.2*0.001 = 3.1164 V

Mismatch = 3.6 mV  →  the EKF thinks "my SOC is too low" and pushes SOC up.
```

EKF SOC guess	Predicted voltage	Mismatch with the real 3.120 V
0.50	3.1164 V	3.6 mV, push up
0.52	3.1184 V	1.6 mV, push up
0.54	3.1204 V	-0.4 mV, about zero (stops)

```
SOC offset = voltage gap / curve slope = 0.0036 V / 0.1 V per unit SOC = about 0.036
```

Once predicted and measured voltage match, the mismatch is zero and the correction stops, even though SOC is still about 0.04 too high. **A tiny voltage error (millivolts) becomes a noticeable SOC error because the curve is flat.** On a steep curve the same few millivolts would change SOC by only a fraction of a percent, which is why Test 1 had almost no offset.

Analogy: measuring a table with a ruler that is 2% too short. The EKF is confident because its measurement matches what its ruler says, but its ruler (the biased current) is slightly off.

Prove it: with no current bias (`Im = 1.00*I + 0.5*randn(1,N)`), the leftover offset in the flat region drops to about 0.2%.

(These numbers use steady-state values and ignore noise, so they are approximate.)

Q. What does the top graph of Test 2 show?

Three lines, each an answer to "how much battery is left?": the **true SOC** (0.8 down to 0.2), **Coulomb counting** (stuck about 0.2 too low, ending near 0%), and the **EKF** (jumps to about 0.8 at the start and then follows the truth, slightly above it). If the dashboard used Coulomb counting it would show about 0% while the real battery still has 20%.

Tuning the EKF

If you...	Then the EKF...
Increase Rn	Trusts the sensor less: smoother but slower to correct
Decrease Rn	Trusts the sensor more: corrects fast but is noisier
Increase Qn	Trusts the model less: follows the sensor more
Decrease Qn	Trusts the model more: smoother
Start with a large P	Allows large early corrections

Experiments worth trying: start in the flat region (done above), raise `Rn` by 100x, raise `Qn(1,1)` to `1e-6`, and use a 10% current bias.

How real BMS products do it

Many combine methods: Coulomb counting all the time, voltage-based correction after the car rests (rested voltage equals OCV), a Kalman-type filter while driving, and recalibration near full charge or empty.

Key takeaways (Section 2)

- No sensor measures SOC, so the BMS estimates it.
- Coulomb counting drifts and cannot fix a wrong start. The EKF adds voltage feedback through a battery model, so it self-corrects.
- The EKF's correction strength depends on the **slope of the OCV curve**: strong at the steep ends, weak in the flat middle of LFP.
- Accuracy in the flat region depends heavily on current-sensor accuracy.

3. Passive Cell Balancing

3.1 The problem

An EV pack has many cells in series, and the same current flows through all of them. The cells are never identical, so their SOC drifts apart.

Analogy: four water bottles filled to different levels from the same tap.

- When **charging**, the fullest bottle overflows first, so you must stop while the others still have room.
- When **discharging**, the emptiest bottle runs dry first, so you must stop while the others still have water.

The pack can only use the range between the emptiest and the fullest cell. A big spread wastes capacity and risks over-charging the fullest cell.

Q. What is the "8%"?

The spread: highest SOC minus lowest SOC.

Cell	SOC
Cell 1	0.98
Cell 3	0.95
Cell 2	0.92
Cell 4	0.90

```
Spread = 0.98 - 0.90 = 0.08  →  8%
```

3.2 Passive vs. active balancing

	Passive	Active
How	Burn the extra charge of the high cells as heat in a resistor	Move charge from high cells to low cells
Cost	Cheap and simple	More expensive
Efficiency	Wastes energy	Saves energy
Used in	Most student packs, many production EVs	Premium packs

3.3 How bleeding works

Each cell has a switch (usually a MOSFET controlled by the BMS) in series with a bleed resistor `Rb`.



- Switch ON: current flows through the resistor, removing charge from the cell as heat.
- Switch OFF: the cell is left alone.

Setup used: 4 cells of 10 Ah (so the demo takes hours, not days), `Rb = 10 Ω`, time step 10 s, 4 hours simulated, starting SOC `[0.98 0.92 0.95 0.90]`, rested cells (so each cell's voltage equals its OCV).

The rule: every cell whose voltage is more than the dead band above the **lowest** cell bleeds. The lowest cell never bleeds.

3.4 Core code

```
Rb    = 10;       % bleed resistor [ohm]
dV_on = 0.005;    % bleed a cell if it is 5 mV above the lowest cell

for k = 1:N
    V(:,k)     = OCV(SOC(:,k));            % at rest, voltage = OCV
    Vmin       = min(V(:,k));              % lowest cell is the reference
    bleed(:,k) = (V(:,k) - Vmin) > dV_on;  % 1 = bleeding ON for that cell
    if k < N
        Ib = bleed(:,k) .* V(:,k) / Rb;            % bleed current [A]
        SOC(:,k+1) = SOC(:,k) - Ib*dt ./ Q;        % remove charge
    end
end
```

3.5 The numbers

Initial voltages (steep region, slope 0.7 V per unit SOC, which is 7 mV per 1% SOC):

Cell	SOC	Voltage	Above the lowest cell
Cell 1	0.98	3.38 + 0.08 × 0.7 = 3.436 V	56 mV
Cell 3	0.95	3.38 + 0.05 × 0.7 = 3.415 V	35 mV
Cell 2	0.92	3.38 + 0.02 × 0.7 = 3.394 V	14 mV
Cell 4	0.90	3.380 V	0 mV (lowest)

Bleed current and rate:

```
current = V / Rb = 3.4 V / 10 Ω = 0.34 A
SOC drop per hour = 0.34 Ah / 10 Ah = 0.034 = about 3.4% per hour
```

Because the current is constant, the SOC lines are **straight**. The heat per bleeding cell is about `3.4 V × 0.34 A = 1.2 W`.

Q. What is the dead band, and why did the lines stop at 0.907 instead of 0.90?

The problem it solves: if the rule were "bleed if even slightly above the lowest cell", the switch would turn ON and OFF hundreds of times as the difference hovers near zero. **Analogy: an air conditioner** that turns on at 25.0 °C and off at 24.9 °C would click constantly, so it uses a small buffer zone instead.

In the code: `dV_on = 0.005` (5 mV). A cell bleeds only while it is more than 5 mV above the lowest cell.

```
Lowest cell (cell 4) = 3.380 V and never changes (it never bleeds)
Bleeding continues while V > 3.380 + 0.005 = 3.385 V
Bleeding stops when V reaches 3.385 V
```

Which SOC is 3.385 V?

```
SOC = 0.90 + (3.385 - 3.380) / 0.7 = 0.90 + 0.0071 = 0.9071
```

So the cells stop at **0.907**, not 0.900. The leftover 0.7% is the width of the dead band expressed in SOC. A smaller dead band gives tighter balance (1 mV would stop at about 0.9014) but more switching. Setting `dV_on = 0.02` would stop at about `0.90 + 0.02/0.7 = 0.929`.

Q. How long does each cell bleed?

`time = (starting SOC - 0.9071) / (about 3.4% per hour)`

Cell	Must drop	Calculation	Bleed time
Cell 2	0.92 to 0.9071 = 1.29%	1.29 / 3.39	about **0.38 h**
Cell 3	0.95 to 0.9071 = 4.29%	4.29 / 3.40	about **1.26 h**
Cell 1	0.98 to 0.9071 = 7.29%	7.29 / 3.41	about **2.14 h**
Cell 4	0		**never**

(The rate differs slightly per cell because the current V/Rb depends on each cell's own voltage.) Check: cell 1 bleeds 2.14 h at about 0.34 A, removing 0.73 Ah, which is 7.3% of 10 Ah.

Q. How do I read the bottom graph (bleed ON/OFF)?

Each cell has its own line. When the line is raised one step above its label (C1, C2, ...), the bleed switch is **ON**. When it sits at its label level, the switch is **OFF**. Applying the rule:

Time	Cell 1	Cell 2	Cell 3	Cell 4
0 h	ON (56 mV)	ON (14 mV)	ON (35 mV)	OFF
0.5 h	ON (about 44 mV)	OFF (5 mV, not above 5)	ON (about 23 mV)	OFF
after 2.2 h	OFF	OFF	OFF	OFF

Phases: all three high cells bleed (0 to 0.4 h), then cells 1 and 3 (to 1.3 h), then only cell 1 (to 2.1 h), then everyone is within the dead band.

3.6 Results

- SOC spread: about **8% to 0.7%**
- Voltage spread: **56 mV to about 5 mV**
- Cell 4 (lowest) never bleeds; cell 1 (highest) bleeds longest

Why balance at the top of the charge?

Near 90 to 100% SOC the LFP curve is steep (about 7 mV per 1% SOC), so voltage differences between cells are visible. In the flat middle region, voltage differences are too small to rely on, so real BMS designs balance near the top of charge.

Limitations worth stating

1. The demo uses 10 Ah cells so it finishes in hours. With 60 Ah cells and the same bleed current, balancing takes about 6 times longer. Real passive balancing is slow, which is why active balancing exists.
2. Passive balancing wastes the removed energy as heat.
3. All cells have the same capacity here. Real cells also differ in capacity and resistance.

Key takeaways (Section 3)

- Cells in series drift apart, and the weakest and fullest cells limit the whole pack.
- Passive balancing bleeds the higher cells down to the lowest cell.
- A dead band prevents switch chattering and sets how close the cells end up.
- Balancing is done where the OCV curve is steep, so voltage is a reliable signal.

4. Protection Logic

4.1 What it is

The BMS acts as a safety guard. It watches voltage, current and temperature all the time, and reduces or cuts current when something goes outside its safe range.

Danger	What goes wrong	BMS action
Over-voltage	Cell damage, fire risk	Stop charging
Under-voltage	Permanent capacity loss	Stop discharging
Over-current	Overheating, damage	Limit the current
Over-temperature	Fast aging, fire risk	Derate gradually, then cut

Limits used in the simulation

Limit	Value	Note
Over-voltage	3.65 V	Typical LFP maximum
Under-voltage	2.80 V	Datasheets often say 2.5 V; 2.8 V is used because the placeholder curve falls steeply near empty
Max discharge current	150 A	
Max charge current	100 A	
Derating starts	45 °C	
Hard cut	60 °C	

4.2 Derating

Reduce the allowed current gradually as temperature rises, instead of switching off at once (like a car slowing down as the engine overheats).

```
f = (60 - T) / (60 - 45)        limited between 0 and 1
allowed current = max current × f
```

Temperature	f	Allowed discharge (max 150 A)
45 °C or below	1.00	150 A
50 °C	0.67	100 A
52 °C	0.53	80 A
60 °C	0	0 A

4.3 Latched faults

For over-voltage, under-voltage and the hard temperature cut, once a fault trips it **stays tripped** until reset. Otherwise the system would turn on, trip again, and repeat as the voltage bounces back after the current is cut.

4.4 The thermal model (Cth and Rth)

A simple lumped model: the cell heats from its own losses and loses heat to the surroundings.

```
T(k+1) = T(k) + dt/Cth * ( P - (T(k) - Tamb)/Rth )
P = I² * R0 + U1² / R1          (heat generated, in watts)
```

Analogy: a water tank with a drain.

```
 heat generated --> [ tank ] --> narrow pipe --> surroundings
  (P in watts)     (Cth: size)    (Rth: how narrow)
                   water level = temperature
```

- **Cth (J/K):** a big tank (heavy cell) heats and cools slowly. Roughly mass × specific heat; a 60 Ah LFP cell is on the order of 1000 to 1500 J/K.
- **Rth (K/W):** a narrow pipe (poor cooling) means a higher final temperature. Liquid cooling means a small Rth.

Demo values: `Cth = 600 J/K`, `Rth = 1.2 K/W`, `Tamb = 30 °C` (smaller Cth so the demo heats in minutes).

Numbers at 120 A:

```
P = 120² × 0.002 + 0.12² / 0.001 = 28.8 + 14.4 = 43.2 W
Initial warming rate (no cooling yet) = 43.2 / 600 = 0.072 °C/s (about 4.3 °C/min)
At T = 45 °C: heat leaving = (45-30)/1.2 = 12.5 W, so net 30.7 W
Without protection it would settle at 30 + 43.2 × 1.2 = about 82 °C
Time constant = Cth × Rth = 600 × 1.2 = 720 s
```

4.5 Core code (one time step)

```
% (a) what the BMS measures now
V_meas = OCV(soc(k)) - I_prev*R0 - U1(k);

% (b) check for faults (latched)
if flt == 0
    if     V_meas > V_ov,  flt = 1;      % over-voltage
    elseif V_meas < V_uv,  flt = 2;      % under-voltage
    elseif T(k) >= T_cut,  flt = 3;      % over-temperature
    end
end

% (c) temperature derating factor
f = min(max((T_cut - T(k)) / (T_cut - T_derate), 0), 1);

% (d) allowed current = request limited by the rules
I_ok(k) = min(max(I_req(k), -I_chg_max*f), I_dis_max*f);
if flt > 0, I_ok(k) = 0; end             % fault: open the contactor

% (e) record, then (f) update SOC, U1 and temperature as in Sections 1 and 4.4
```

Q. Why -80 A? What does the sign mean?

The sign is a convention: **positive = discharging, negative = charging**. So `-80 A` means "charge the battery at 80 A". The same equations work for both directions:

- `SOC - (dt/Q) × (-80)` gives SOC **up**, and `OCV - (-80) × R0` gives voltage **up** by 0.16 V (charging).
- With `+60 A`: SOC goes down and voltage goes down (discharging).

Some tools use the opposite convention, so always check the datasheet or model. In `I_ok = min(max(I_req, -I_chg_max*f), I_dis_max*f)`, the BMS squeezes the request between -100 A (charge limit) and +150 A (discharge limit).

Q. What does "scenario 2, 60 A" mean?

+60 A is a **discharge** (like driving), starting from a nearly empty battery (SOC 0.15).

Scenario	Current	Direction	Start SOC	Represents
1. Over-temperature	+120 A	Discharge	100%	Hard driving
2. Under-voltage	+60 A	Discharge	15%	Driving on a nearly empty battery
3. Over-voltage	-80 A	Charge	90%	Fast charging

Each scenario triggers one protection at a time.

4.6 Scenario 1: over-temperature (120 A drive, derating)

- Temperature rises from 30 °C. At about **260 s** it crosses 45 °C and derating starts.
- The allowed current falls from 120 A and settles at about **78 A**, and the temperature levels off at about **52 °C**. **No fault trips**; the hard cut at 60 °C is never needed.

Why 52 °C and 78 A (the balance):

```
At 78 A:  heat produced = 78² × 0.002 + 0.078²/0.001 = 12.2 + 6.1 = 18.3 W
          heat leaving  = (52.2 - 30) / 1.2 = 18.5 W        → equal, so temperature holds
Check:    f = (60 - 52.2)/15 = 0.52  →  150 × 0.52 = 78 A
```

It is a feedback loop: hotter → lower f → less current → less heat → temperature stops rising. The cost is about 35% less power.

Experiment: poor cooling (`Rth = 50`). Almost no heat escapes, so the temperature creeps toward about 58 °C and the allowed current keeps falling to about 18 A at 1500 s. No fault trips, but only about 15% of the requested power remains. Derating keeps the cell safe; **cooling keeps it useful**. The hard cut never triggers because as T approaches 60 °C, f approaches 0, so heating nearly stops. The hard cut is a backup for failures derating cannot catch (for example a sensor fault).

4.7 Scenario 2: under-voltage (60 A at low SOC)

Time	Voltage	What happens
0 s	about 3.12 V	Start (SOC 0.15, minus the load drop)
0 to about 170 s	falls slowly to about 3.02 V	Normal discharge
about 170 s	the line bends downward	SOC about 0.10, the curve gets steep below 10% (the "knee")
about 379 s	reaches 2.8 V	**Under-voltage fault**

```
SOC at 170 s = 0.15 - (60 × 170)/216,000 = 0.103
Charge used by 379 s = 60 × 379 = 22,740 A·s = 6.3 Ah = 10.5% of 60 Ah
SOC at the cut = 15% - 10.5% = about 4.5%
```

- The current goes **straight from 60 A to 0** (a hard cut). The driver still requests 60 A, but the BMS ignores it.
- The voltage then jumps to about 2.98 V: instantly because the R0 drop (60 × 0.002 = 0.12 V) disappears, then slowly as the RC part relaxes. The fault **stays latched** even though the voltage recovered.
- The cell is not truly empty (about 4.5% left). The loaded voltage reaches the limit first, which acts as a safety margin.
- Starting from SOC 0.5 instead, the fault would trip **later** (more charge to use).

4.8 Scenario 3: over-voltage (80 A fast charge)

Charging pushes the terminal voltage **above** the cell's own voltage:

```
Start: 3.38 + 80 × 0.002 = 3.54 V
Later (RC built up): extra push = 80 × (0.002 + 0.001) = 0.24 V
At 117 s: charge added = 80 × 117/216,000 = 4.3%  →  SOC = about 0.943
          OCV = 3.38 + 0.043 × 0.7 = 3.41 V     →  Vt = 3.41 + 0.24 = 3.65 V (the limit)
```

- At about **117 s** the over-voltage fault trips and the current goes from -80 A to 0.
- The voltage drops instantly by 0.16 V (R0 push gone) to about 3.49 V, then relaxes to about **3.41 V**, the true OCV. The fault stays latched.
- **Key insight:** the cell was only at about **94% SOC** when the fault tripped. The voltage limit was hit because of the charging IR push (about 0.24 V), not because the cell was full. A hard cut leaves about 6% capacity unused.
- **How real chargers avoid this: CC-CV charging.** In the constant-current phase the charger pushes a fixed current until the terminal voltage reaches the limit. In the constant-voltage phase it holds the voltage and lets the current **taper**, so the IR push shrinks and the cell fills completely. This is built in Part 4.

4.9 Summary of the three scenarios

Scenario	Trigger	BMS action	Lesson
1. Over-temperature	Hot cell at 120 A	Gradual derating, no fault	Safety costs power, so cooling matters
2. Under-voltage	Vt below 2.8 V at about 4.5% SOC	Hard cut, latched	Not truly empty: a safety margin
3. Over-voltage	Vt above 3.65 V at about 94% SOC	Hard cut, latched	The IR push trips it early, hence CC-CV

Limitations

- Real EVs reduce power gradually near low voltage instead of cutting suddenly (voltage-based derating could be added).
- Real BMS designs add a short debounce time before tripping, so a noise spike does not trip a fault. This simulation has no sensor noise in the protection test.
- Cth and Rth are placeholders; they can be replaced by the Part 2 thermal model.

Key takeaways (Section 4)

- Protection checks voltage, current and temperature every step and limits or cuts the current.
- Temperature uses gradual derating (a feedback loop), voltage faults use hard latched cuts.
- Charging and discharging are the same equations with opposite current signs.
- Protection keeps the battery **safe**; cooling keeps it **useful**.

5. How the Four Parts Fit Together

```
                +--> EKF (Section 2): estimates SOC of each cell
 Thevenin model -|
 (Section 1)    +--> Balancing (Section 3): uses cell voltages (OCV from SOC)
                |
                +--> Protection (Section 4): uses Vt, current and temperature
```

- The **Thevenin model** is the foundation: every other part uses its equations or its OCV table.
- The **EKF** answers "how much charge is left?"
- **Balancing** answers "are the cells equal?"
- **Protection** answers "is it safe to continue?"

Next in the roadmap: Part 4 (CC-CV charging, which fixes the early cut-off in scenario 3), Part 5 (motor control with FOC), Part 6 (everything combined into one simulator).

6. Quick-Revision Formula Sheet

```
SOC update:        SOC(k+1) = SOC(k) - (η*dt/Q) * I
RC voltage:        U1(k+1)  = exp(-dt/τ)*U1(k) + R1*(1-exp(-dt/τ))*I
Terminal voltage:  Vt       = OCV(SOC) - I*R0 - U1
Interpolation:     V = V_low + (SOC - SOC_low) * slope
EKF H:             H = [dOCV/dSOC, -1]
Kalman gain:       K = P*H' / (H*P*H' + Rn)
Correction:        x = x_pred + K * (Vt_measured - Vt_pred)
Offset on flat curve: SOC error = voltage error / OCV slope
Bleed current:     I = V / Rb
Balancing stop:    V = V_lowest + dV_on
Derating:          f = (T_cut - T)/(T_cut - T_derate), allowed I = I_max * f
Thermal model:     T(k+1) = T(k) + dt/Cth * (I²*R0 + U1²/R1 - (T - Tamb)/Rth)
Time constants:    τ = R1*C1 (electrical), Cth*Rth (thermal)
Sign convention:   +I discharge, -I charge
```

7. Self-Test Questions

1. Why does the terminal voltage drop instantly when the current rises, and then keep sagging slowly?
2. Why can't Coulomb counting recover from a wrong starting SOC?
3. What does the "Extended" in EKF refer to?
4. Why does the EKF correct more strongly at 95% SOC than at 50% on an LFP cell?
5. Why does a 2% current bias leave a larger SOC error on a flat OCV curve than on a steep one?
6. In balancing, why does the lowest cell never bleed?
7. What problem does the dead band solve, and what does a larger dead band do to the final spread?
8. Why did the cells stop at 0.907 instead of 0.900?
9. Why is a temperature fault handled by derating but a voltage fault by a hard cut?
10. Why did the over-voltage fault trip when the cell was only at about 94% SOC, and how does CC-CV solve it?

Answers (short)

1. R0 acts instantly (voltage = current × R0). The RC pair charges gradually, which creates the slow sag.
2. It has no feedback from any measurement, so any error stays forever.
3. It linearises the curved OCV-SOC relationship at the current operating point.
4. The OCV curve is steep there (H large, K large), so voltage carries a clear SOC signal. In the flat middle it is below the sensor noise.
5. The EKF settles where predicted and measured voltage match. A few mV of mismatch divided by a small slope gives a large SOC shift.
6. It is the reference: nothing is above it, so there is nothing to remove.
7. It prevents switch chattering. A larger dead band means a larger leftover spread.
8. Bleeding stops when a cell is within 5 mV of the lowest cell (3.385 V), which corresponds to SOC 0.9071.
9. Temperature changes slowly and responds to current, so a feedback loop can hold it below the limit. Voltage faults are abrupt and are cut immediately.
10. The charging IR push (about 0.24 V) lifts the terminal voltage to the limit before the cell is full. CC-CV holds the voltage and tapers the current so the push shrinks and the cell can fill completely.